

ASSIGNMENT – 6.4

2303A51123

Batch-10

Task-1

Prompt: generate a Python class Student with name, roll_number, and marks. Add a method to display details and another method to check if marks are above the class average using if-else. take user input

Code :

```
class Student:
    def __init__(self, name, roll_number, marks):
        self.name = name
        self.roll_number = roll_number
        self.marks = marks
    def display_details(self):
        print(f'Name: {self.name}')
        print(f'Roll Number: {self.roll_number}')
        print(f'Marks: {self.marks}')
    def is_above_average(self, average_marks):
        if self.marks > average_marks:
            return f'{self.name} is above the class average.'
        else:
            return f'{self.name} is not above the class average.'

students = []
num_students = int(input("Enter the number of students: "))
for _ in range(num_students):
    name = input("Enter student's name: ")
    roll_number = input("Enter student's roll number: ")
    marks = float(input("Enter student's marks: "))
    students.append(Student(name, roll_number, marks))
total_marks = sum(student.marks for student in students)
average_marks = total_marks / num_students
print(f"\nClass Average Marks: {average_marks:.2f}\n")
for student in students:
```

```

student.display_details()
print(student.is_above_average(average_marks))
print()

```

Output :

The screenshot shows a VS Code editor with a file named 'lab-6.4.py' open. The code defines a 'Student' class with attributes 'name', 'roll_number', and 'marks'. It includes methods 'display_details' and 'is_above_average'. A list 'students' is initialized. The terminal output shows the execution of the script, where user input is used to create a student object and calculate the class average.

```

1 #Task-1
2 #generate a Python class Student with name, roll_number, and marks. Add a method to display details and another
3 class Student:
4     def __init__(self, name, roll_number, marks):
5         self.name = name
6         self.roll_number = roll_number
7         self.marks = marks
8     def display_details(self):
9         print(f"Name: {self.name}")
10        print(f"Roll Number: {self.roll_number}")
11        print(f"Marks: {self.marks}")
12    def is_above_average(self, average_marks):
13        if self.marks > average_marks:
14            return f"{self.name} is above the class average."
15        else:
16            return f"{self.name} is not above the class average."
17
18 students = []

```

```

PS C:\Users\hp\OneDrive\Desktop\AI> & C:/Users/hp/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/hp/OneDrive/D
.py
Enter the number of students: 1
Enter student's name: vamshi
Enter student's roll number: 1123
Enter student's marks: 80

Class Average Marks: 80.00

Name: vamshi
Roll Number: 1123
Marks: 80.0
vamshi is not above the class average.

```

Code Analysis :

- ☐ Defines a Student class with name, roll number, and marks as attributes.
- ☐ Uses methods to display student details and compare marks with class average.
- ☐ Takes user input to create multiple student objects and stores them in a list.
- ☐ Calculates class average using total marks and number of students.
- ☐ Uses if-else to check whether a student is above average

Task-2

Prompt: Write a for loop to iterate through sensor readings, identify even numbers using modulus operator, calculate their square, and print the result clearly and user input.

Code :

```

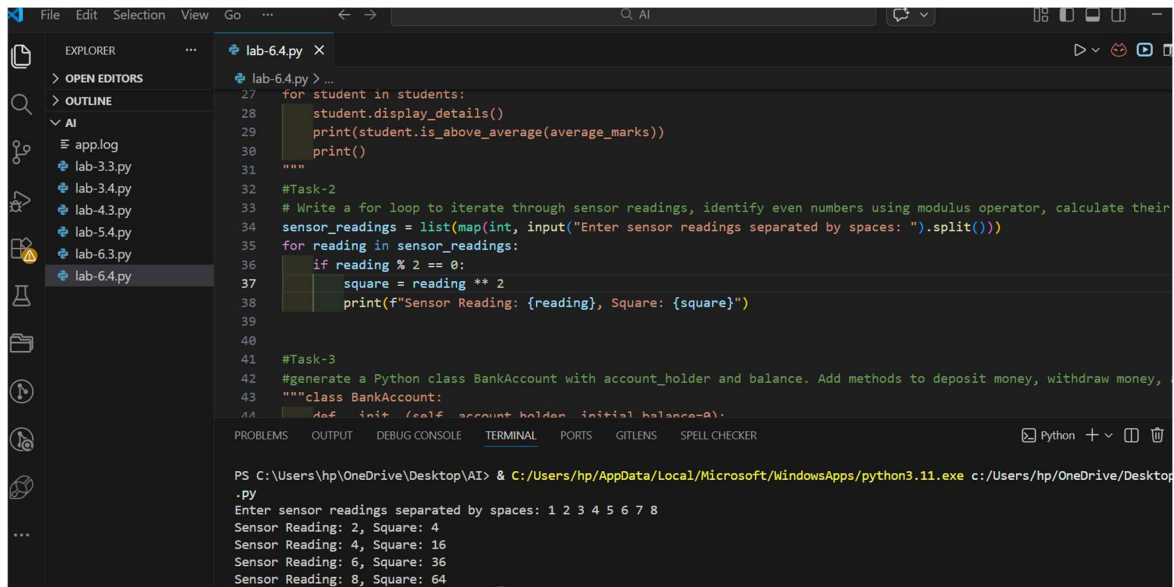
sensor_readings = list(map(int, input("Enter sensor readings separated by spaces: ").split()))
for reading in sensor_readings:
    if reading % 2 == 0:

```

```
square = reading ** 2
```

```
print(f'Sensor Reading: {reading}, Square: {square}')
```

Output :



The screenshot shows a VS Code editor with a file named 'lab-6.4.py' open. The code in the editor includes a for loop for 'Task-2' that iterates through sensor readings, identifies even numbers using the modulus operator, calculates their squares, and prints the results. The terminal output shows the execution of the script, where the user enters sensor readings '1 2 3 4 5 6 7 8' and the program outputs the square for each even reading: 'Sensor Reading: 2, Square: 4', 'Sensor Reading: 4, Square: 16', 'Sensor Reading: 6, Square: 36', and 'Sensor Reading: 8, Square: 64'.

Code Analysis :

- ☐ Takes multiple sensor readings as user input in a list format.
- ☐ Uses a for loop to iterate through each sensor reading.
- ☐ Applies the modulus operator (%) to identify even numbers.
- ☐ Calculates the square of even readings using the exponent operator.
- ☐ Prints the reading and its square in a clear format.

Task-3

Prompt: generate a Python class Bank Account with account_holder and balance. Add methods to deposit money, withdraw money, and prevent withdrawals when balance is insufficient using if-else user input.

Code :

```

class BankAccount:
    def __init__(self, account_holder, initial_balance=0):
        self.account_holder = account_holder
        self.balance = initial_balance
    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: ${amount}")
        else:
            print("Deposit amount must be positive.")
    def withdraw(self, amount):
        if 0 < amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew: ${amount}")
        else:
            print("Insufficient balance or invalid withdrawal amount.")
    def check_balance(self):
        print(f"Current balance: ${self.balance}")

account_holder = input("Enter account holder's name: ")
initial_balance = float(input("Enter initial balance: "))
account = BankAccount(account_holder, initial_balance)

while True:
    action = input("Choose an action: deposit, withdraw, check balance, or exit: ").lower()
    if action == "deposit":
        amount = float(input("Enter amount to deposit: "))
        account.deposit(amount)
    elif action == "withdraw":
        amount = float(input("Enter amount to withdraw: "))
        account.withdraw(amount)

```

```

elif action == "check balance":

    account.check_balance()

elif action == "exit":

    print("Exiting the program.")

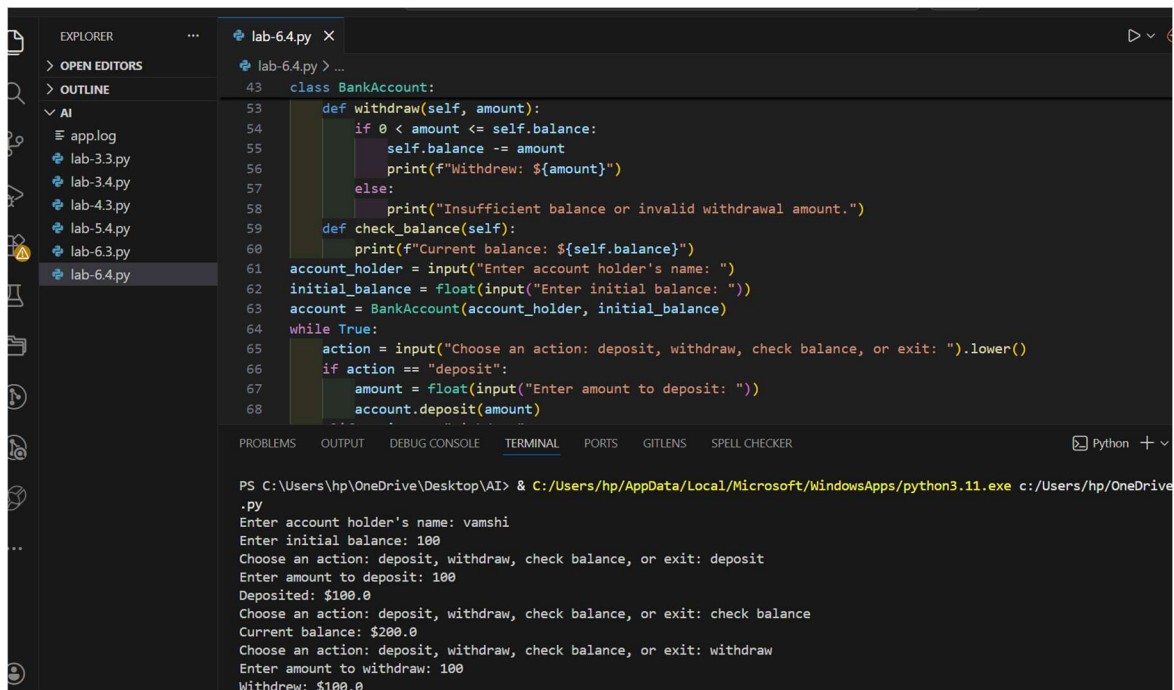
    break

else:

    print("Invalid action. Please choose again.")

```

Output :



```

lab-6.4.py
43 class BankAccount:
44     def withdraw(self, amount):
45         if 0 < amount <= self.balance:
46             self.balance -= amount
47             print(f"Withdrew: ${amount}")
48         else:
49             print("Insufficient balance or invalid withdrawal amount.")
50     def check_balance(self):
51         print(f"Current balance: ${self.balance}")
52 account_holder = input("Enter account holder's name: ")
53 initial_balance = float(input("Enter initial balance: "))
54 account = BankAccount(account_holder, initial_balance)
55 while True:
56     action = input("Choose an action: deposit, withdraw, check balance, or exit: ").lower()
57     if action == "deposit":
58         amount = float(input("Enter amount to deposit: "))
59         account.deposit(amount)

```

```

PS C:\Users\hp\OneDrive\Desktop\AI> & C:/Users/hp/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/hp/OneDrive
.py
Enter account holder's name: vamshi
Enter initial balance: 100
Choose an action: deposit, withdraw, check balance, or exit: deposit
Enter amount to deposit: 100
Deposited: $100.0
Choose an action: deposit, withdraw, check balance, or exit: check balance
Current balance: $200.0
Choose an action: deposit, withdraw, check balance, or exit: withdraw
Enter amount to withdraw: 100
Withdrew: $100.0

```

Code Analysis :

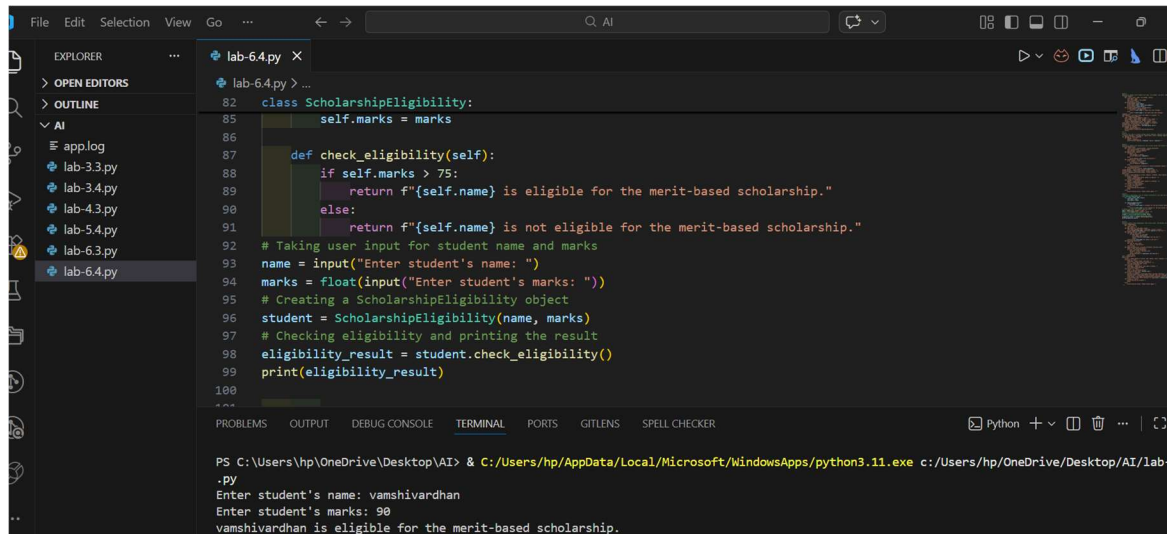
- ☐ Creates a BankAccount class with account holder name and balance.
- ☐ Includes methods for deposit, withdrawal, and balance checking.
- ☐ Uses if-else to prevent withdrawals with insufficient balance.
- ☐ Accepts user input through a menu-driven while loop.
- ☐ Ensures valid banking operations and safe program exit.

Task-4

Prompt: generate a code Using a list of student dictionaries with name and score, write a class ScholarshipEligibility:

```
def __init__(self, name, marks):  
    self.name = name  
    self.marks = marks  
  
def check_eligibility(self):  
    if self.marks > 75:  
        return f'{self.name} is eligible for the merit-based scholarship.'  
    else:  
        return f'{self.name} is not eligible for the merit-based scholarship.'  
  
# Taking user input for student name and marks  
name = input("Enter student's name: ")  
marks = float(input("Enter student's marks: "))  
  
# Creating a ScholarshipEligibility object  
student = ScholarshipEligibility(name, marks)  
  
# Checking eligibility and printing the result  
eligibility_result = student.check_eligibility()  
print(eligibility_result)
```

Output :



```
File Edit Selection View Go ...
lab-6.4.py x
lab-6.4.py > ...
82 class ScholarshipEligibility:
83     self.marks = marks
84
85     def check_eligibility(self):
86         if self.marks > 75:
87             return f"{self.name} is eligible for the merit-based scholarship."
88         else:
89             return f"{self.name} is not eligible for the merit-based scholarship."
90
91 # Taking user input for student name and marks
92 name = input("Enter student's name: ")
93 marks = float(input("Enter student's marks: "))
94 # Creating a ScholarshipEligibility object
95 student = ScholarshipEligibility(name, marks)
96 # Checking eligibility and printing the result
97 eligibility_result = student.check_eligibility()
98 print(eligibility_result)
99
100
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER
Python + - [ ] [ ] [ ] [ ]
P5 C:\Users\hp\OneDrive\Desktop\AI> & C:/Users/hp/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/hp/OneDrive/Desktop/AI/lab-6.4.py
Enter student's name: vamshivardhan
Enter student's marks: 90
vamshivardhan is eligible for the merit-based scholarship.
```

Code Analysis :

- ☐ Defines a class to store student name and marks.
- ☐ Takes user input for student details.
- ☐ Uses an if-else condition to check marks greater than 75.
- ☐ Determines eligibility for a merit-based scholarship.
- ☐ Displays the eligibility result clearly.

Task 5

Prompt: #Create a Python class Shopping Cart that stores items. Add methods to add items, remove items, calculate total using a loop, and apply discount if total exceeds a limit user input.

class ShoppingCart:

def __init__(self):

self.items = []

def add_item(self,

item_name, price):

self.items.append({"na

me": item_name, "price":

price})

```
        print(f'Added
{item_name} with price
${price} to the cart.")
    def remove_item(self,
item_name):
        for item in self.items:
            if item["name"] ==
item_name:
                self.items.remove
(item)
                print(f'Removed
{item_name} from the
cart.")
            return
        print(f'Item
{item_name} not found in
the cart.")
    def calculate_total(self):
        total = 0
        for item in self.items:
            total +=
item["price"]
        return total
    def apply_discount(self,
discount_threshold,
discount_rate):
        total =
self.calculate_total()
```



```

        if total >
discount_threshold:
            discount = total *
discount_rate
            total -= discount
            print(f'Discount of
${discount:.2f} applied.")
            return total
cart = ShoppingCart()
while True:
    action = input("Choose
an action: add, remove,
total, checkout, or exit:
").lower()
    if action == "add":
        item_name =
input("Enter item name: ")
        price =
float(input("Enter item
price: "))
        cart.add_item(item_na
me, price)
    elif action == "remove":
        item_name =
input("Enter item name to
remove: ")
        cart.remove_item(item
_name)
    elif action == "total":

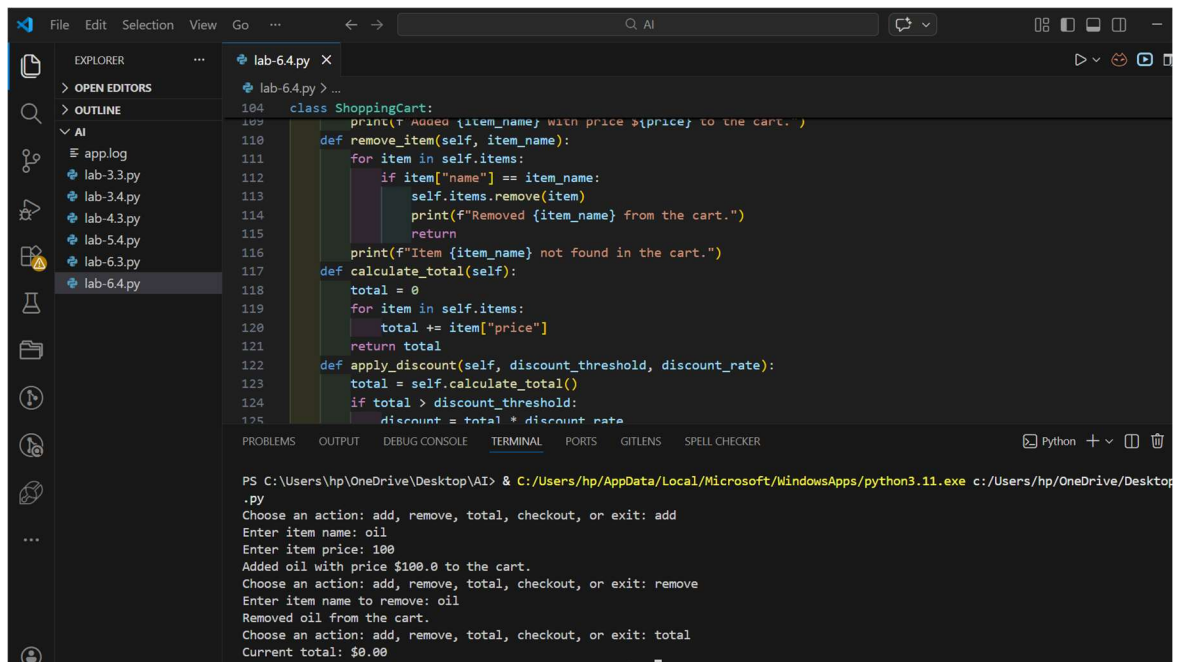
```

```

        total =
    cart.calculate_total()
        print(f"Current total:
    ${total:.2f}")
        elif action ==
    "checkout":
            discount_threshold =
    float(input("Enter discount
    threshold: "))
            discount_rate =
    float(input("Enter discount
    rate (as a decimal): "))
            final_total =
    cart.apply_discount(discou
    nt_threshold,
    discount_rate)
            print(f"Final total after
    discount (if applicable):
    ${final_total:.2f}")
            elif action == "exit":
                print("Exiting the
    program.")
                break
            else:
                print("Invalid action.
    Please choose again.")

```

Output :



The screenshot shows a VS Code editor with a file explorer on the left containing several Python files (lab-3.3.py to lab-6.4.py). The main editor window displays the code for lab-6.4.py, which defines a `ShoppingCart` class. The class has methods for adding items, removing items, calculating the total, and applying a discount. The terminal at the bottom shows the execution of the script, which prompts the user for actions and item details, demonstrating the functionality of the `ShoppingCart` class.

```
104 class ShoppingCart:
105     print(f"Added {item_name} with price ${price} to the cart. ")
110 def remove_item(self, item_name):
111     for item in self.items:
112         if item["name"] == item_name:
113             self.items.remove(item)
114             print(f"Removed {item_name} from the cart.")
115             return
116     print(f"Item {item_name} not found in the cart.")
117 def calculate_total(self):
118     total = 0
119     for item in self.items:
120         total += item["price"]
121     return total
122 def apply_discount(self, discount_threshold, discount_rate):
123     total = self.calculate_total()
124     if total > discount_threshold:
125         discount = total * discount_rate
```

PS C:\Users\hp\OneDrive\Desktop\AI> & C:/Users/hp/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/hp/OneDrive/Desktop/...
_py
Choose an action: add, remove, total, checkout, or exit: add
Enter item name: oil
Enter item price: 100
Added oil with price \$100.0 to the cart.
Choose an action: add, remove, total, checkout, or exit: remove
Enter item name to remove: oil
Removed oil from the cart.
Choose an action: add, remove, total, checkout, or exit: total
Current total: \$0.00

Code Analysis :

- ☐ Implements a `ShoppingCart` class to store items in a list.
- ☐ Provides methods to add and remove items from the cart.
- ☐ Uses a loop to calculate the total price of items.
- ☐ Applies a discount when total exceeds a user-defined threshold.
- ☐ Uses a while loop for continuous user interaction.